

지식 자산화 · 하네스 엔지니어링 실무

AI PRD 와 현업의 문제를 기반으로 에이전트가 신뢰할 수 있는 자산으로 바꾸고,
에이전트의 행동 규칙을 명시적으로 설계한다

8시간 · 4개 모듈

RAG vs 지식그래프 실습

산출물: AGENTS.md · 메모리 파일

트로이(TROE) · 삼성전자 MX 구미사업장 · 2026

박 충 효

APPENDIX & PROFILE

21yr+ 마케터 & 트로이 대표 파트너(Chief Executive Partner, Sotatek JSC & Appier SRA)

troe.kr | linkedin @chunghyopark

디지털 전략 기반의 Full Category 진행(AI X 데이터 기반 콘텐츠 마케팅 의사결정 자문, 분석 등)

AI X '콘텐츠 전략, 개발, 자동화 워크플로우' 중심 주요 기업 트레이닝(삼성전자 인재개발원, 한국 콜마 그룹 외)

(현) 클라우드웍스 아카데미, 교육 파트너(2026.01 ~)

(현) 데이원컴퍼니, 패스트캠퍼스(커널아카데미) AI Agent-AI PM 심화과정 교육 및 실습/상주 코치(2025.10 ~2026.07)

(전) 유니트론테크, 사내 ERP 구축 기반의 AX 프로젝트 수행

(전) 카카오, CTO 조직의 마케팅 전략 구축(AI Native) 및 AI 기술 기반 디지털 콘텐츠 프로젝트

(전) 카카오브레인(카카오 AI 전담 개발 조직), 디지털 콘텐츠 프로젝트

(전) 패스트캠퍼스 Business Analytics & Byte Degrees 등 (마케팅 관점의)실무 데이터 분석, AI 응용 콘텐츠 전략 과정 진행

(전) BKR/버거킹 코리아의 온라인 버즈 분석 프로젝트(with AI 기반)

(전) 한화에어로스페이스, 사내 뉴스레터 콘텐츠 PM

(전) 삼성전자 모바일 사업부 TF의 '광고 플랫폼(프로그래머틱)'의 온보딩 교육 프로젝트 수행

(전) 국내 커머스, 베트남과 일본 크로스보더 커머스 조직 CMO, 마케팅 리드 외 다수 디지털 마케팅 업무 진행



TROE(트로이), Strategy Sales and AI Transformation(AX) Partner with

Appier



8시간 타임테이블

Day 1의 Problem Brief를 팀 프로젝트로 확장하고, 지식·하네스·컨텍스트 3개 실무 축을 완주한다

시간	모듈	내용	활용 도구
08:30-09:30	팀 빌딩 및 진단	역량 진단 워크시트 · 팀 마일스톤 구축	-
09:45-11:30	지식 자산화	RAG vs 지식그래프 비교 · 금형 ECO·기구 문서 변환 실습	Claude Code, Github Repo
11:30-13:00	Lunch	중식 및 휴게 시간	-
13:00-14:45	하네스 설계	하네스 엔지니어링 개념 · AGENTS.md로 행동 지침·제약 통제	NotebookLM, Claude Code, OpenAI Codex
15:00-17:30	컨텍스트 실무	외부 메모리 상태 관리 · 마크다운 기록으로 장기 시나리오 완주	-

1

Day 2 · Module 1 ·

09:00–10:30

오늘 어디서 부터 시작하는가

역량 진단과 트랙 분기 — Standard/Advanced

가볍게 Wrap-up 하고 갈까요?

개인/팀 안에 경험 차이가 크면, 진단 없이 시작할 수록 당초 '목표한 바'까지 도달 할 수 없습니다. 단계별로 하나씩 해나가는 과정 입니다

비유로 설명하면

체육 시간에 다 같이 "오늘부터 마라톤"이라고 하면 안 된다.

누구는 매일 뛰던 사람이고, 누구는 오늘 처음 운동화를 신었을 수 있다.

그래서 먼저 각자의 현재 체력(=프로그래밍 경험)을 확인하고,

그에 맞는 코스(트랙)를 고른 뒤에 함께 출발한다.(오늘 이 자리의 대부분은 비개발자라고 생각하고 AI 도구를 최대한 사용합니다)

쉽게 말하면

오늘 우리 팀이 하는 일: 팀원 각자의 경험을 확인하고,

"쉬운 길(Standard)"과 "직접 설계하는 길(Advanced)" 중 하나를 팀으로 고른다.

프로젝트를 시작하기 전, 각자의 결과물을 스스로 되돌아 봅니다!

Day 1 설문 기준 — 프로그래밍 무경험 22.7%, 코딩 에이전트 경험 17.7%가 한 팀에 섞여 있다

Standard 트랙

템플릿 중심 · 바이브 코딩

코드는 전부 에이전트에게 맡긴다.

"무엇을 해주세요"라고 말로 설명만 하면 된다.

템플릿 저장소를 받아 지시문(프롬프트)만 바뀌가며 진행.

추천 대상 — 프로그래밍 경험이

'전혀 없음'이거나 엑셀 수식·매크로 수준인 팀원이 절반을 넘는 팀

Advanced 트랙

구조 설계 중심 · 직접 확장

템플릿을 뼈대로, AGENTS.md 규칙과

폴더 구조를 팀이 직접 넓혀간다.

에이전트가 "어떻게 일할지"까지우리가 설계한다는 뜻이다.

추천 대상 — 실무에서 코드를 상시

개발하거나 직접 작성 가능한 팀원 (설문 40.9%)이 다수인 팀

각자의 활용 방법, 출력 결과물은 다를 수 있습니다

Standard를 골랐다고 해서 "뒤쳐진 팀"이 아닙니다 — 등산 코스가 다를 뿐 정상은 같다라는 것을 잊지 맙시다!

비유로 설명하면

등산할 때 초급 코스로 오른 사람과 중급 코스로 오른 사람이
같은 정상에서 만나는 것처럼, Standard 트랙과 Advanced 트랙 모두
Day 5에는 "작동하는 자동화 파이프라인 + 리포트"라는
같은 산 정상에서 만난다.

그리고 등산 중에 체력이 붙으면 코스를 바꿀 수 있듯,
트랙도 언제든지 Standard → Advanced로 전환할 수 있다.

GitHub 계정 만들기

1. github.com 접속 → 이름 · 이메일 · 비밀번호로 가입
2. 가입 메일함에서 인증 메일 확인하고 인증 완료
3. (선택) 오른쪽 위 Settings → SSH and GPG keys 에서
내 컴퓨터의 SSH 키 등록

가입 폼 이렇게 채워요

github.com 접속 후

\$ 이름 : 홍길동

\$ 이메일 : hong@example.com

\$ 비밀번호 : ****

\$ → [Sign up] 클릭!



무료 계정도 공개(Public) 저장소는 무제한이에요 — 비공개는 나중에 필요할 때 유료 옵션으로!

Git 저장소 만들기

제일 먼저, 이름과 이메일부터 등록해요 (최초 1회만!)

내 이름표 붙이기

```
$ git config --global user.name "이름"  
$ git config --global user.email "이메일"
```

새로 시작할 때

```
$ git init
```

이미 있을 때

```
$ git clone <  
주소>
```



git init은 폴더만 Git 저장소로 바꿔줄 뿐, 아직 아무 파일도 저장 안 해요 — 다음 페이지(커밋)로 이어져야 진짜 저장!

STEP

3

작업한 내용을 스냅샷으로 저장해요

커밋하기

수정

Working
Directory

담기

Staging
(git add)

저장

Repository
(git commit)

이 세 줄이면 끝이에요

```
$ git status      # 뭐가 바뀌었는지 확인
$ git add .       # 커밋 대기열에 담기
$ git commit -m "메시지" # 스냅샷으로 저장
```

📌 커밋은 게임의 세이브 포인트 같은 거예요 — 나중에 언제든지 그 시점으로 돌아갈 수 있어요.

STEP

4

내 컴퓨터의 저장 내용을 GitHub로 올리요

Push 하기

git init으로 시작했다면, 먼저 원격 주소부터 연결해요

컴퓨터 → GitHub 링크!

```
$ git remote add origin https://github.com/이름/저장소.git  
$ git push -u origin main # 최초 1회, -u로 연결까지
```

두 번째 push부터는

```
$ git push  
거 한 줄!
```

다음부터 이



다른 사람이 먼저 push했다면? git pull로 받아온 뒤에 다시 push!

브랜치 관리

다섯 줄로 끝나는 브랜치

```
$ git checkout -b 새기능      # 만들고 바로
이동
# ...작업하고 커밋...
$ git checkout main          # 원래 자리로
$ git merge 새기능           # 합치기
$ git branch -d 새기능       # 다 썼으면 삭제
```

충돌(Conflict)이 뜨면?

<<<<<<< ===== >>>>>>> 사이 내용을 비교해서

원하는 대로 정리하고, 구분선은 지워요.

→ git add 로 다시 담고 → git commit 으로 마무리!



브랜치는 원본을 건드리지 않고 실험하는 공간이에요 — 실패해도 main은 늘 안전해요. 여기까지 오셨다면 이제 진짜 시작이에요!



이 4가지가 없으면 아직 출발하지 않은 것이다

- ☐ 개인/팀 전체가 Standard/Advanced 중 하나에 합의했는가 (한 사람만 정한 것이 아닌가)
- ☐ Day 5까지 5개 마일스톤에 날짜가 적혀 있는가
- ☐ GitHub 저장소에 팀원 전원이 접근할 수 있는가
- ☐ "역량 진단 워크시트 + 팀 마일스톤" 산출물이 저장소에 올라가 있는가

2

Day 2 · Module 2 ·

11:00–12:00

에이전트는 무엇을 알아야 하는가

지식 자산화 — RAG vs 지식그래프



"대부분의 사람들은 제 글, 트윗, 팟캐스트, 강연 등을 에이전트에게 보여주기만 해도 에이전트의 업무 효율을 100배까지 높일 수 있다는 사실을 모릅니다."

- 라이언 로포폴로(https://x.com/_lopopolo/status/2050698864542482709)
- Harness 를 공개한 OpenAI 초기 개발자(26.02.11)

☆ 지식 자산화와 하네스 이야기 ☆



☁ 엉망인 데이터를 ➡ 지식 자산으로 만들고, ➡ 하네스에서 가치 있게 활용하자! ☆

문서는 있는데, 왜 에이전트는 모를까

문서가 많아도 출처·최신성·맥락·사용 조건이 없으면 아직 지식자산이 아니다

비유로 설명하면

창고에 상자가 많아도 목록과 위치표가 없으면 필요한 물건을 꺼낼 수 없다.

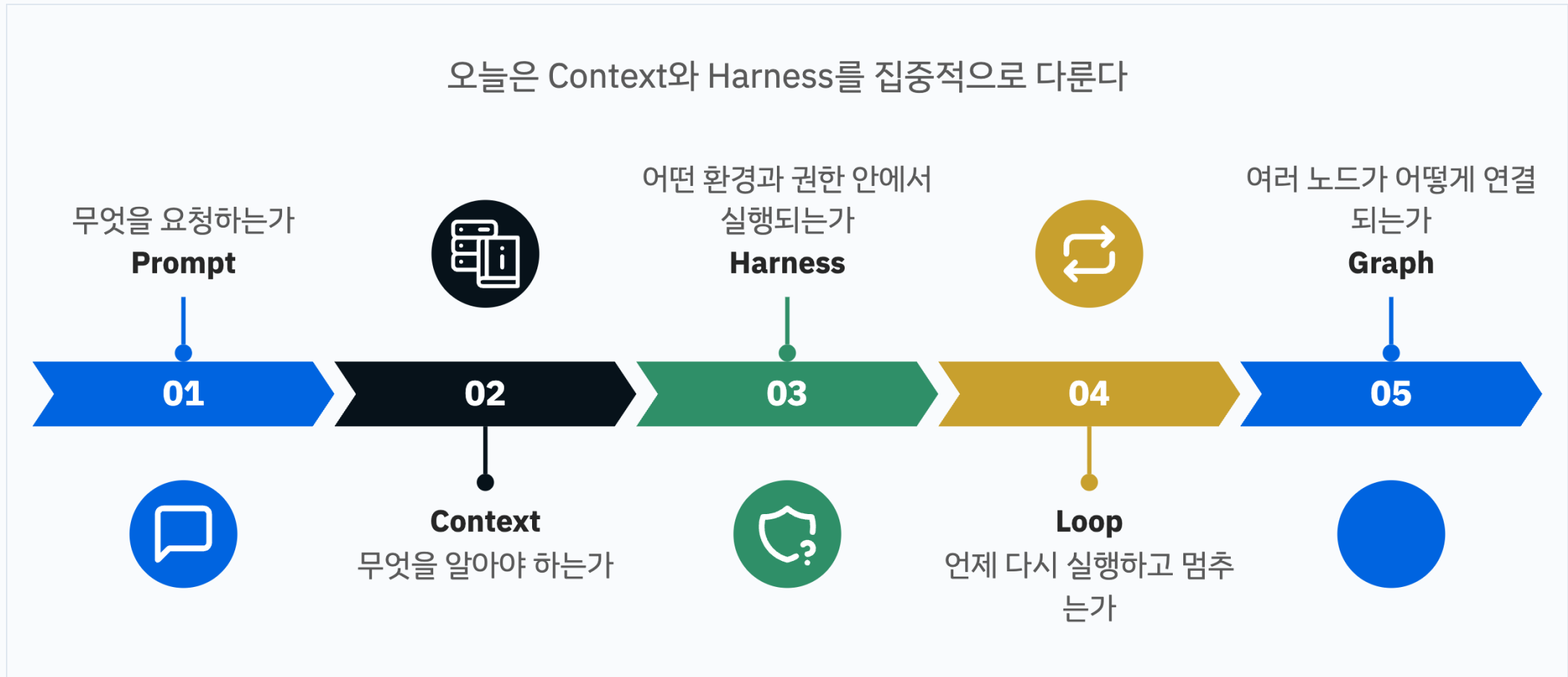
문서도 마찬가지다. 만든 사람·작성 시점·업무 목적·최신본 여부·사실과 의견·관계·권한이 붙어야 다시 쓸 수 있다.

지식자산화는 자료를 더 쌓는 일이 아닌,

에이전트가 근거를 확인하고 재사용할 수 있게 구조와 맥락을 붙이는 일이다.

Day 1에서 배운 위치를 다시 확인한다

오늘은 Context 계층(무엇을 알아야 하는가)과 Harness 계층(어떤 환경 안에서 실행되는가)을 다룬다



(기초에 대한 이해부터) RAG(벡터 검색)란 무엇인가

"검색 증강 생성" — 문서를 통째로 외우지 않고, 필요할 때 관련 부분만 찾아서 답한다

비유로 설명하면

도서관에 처음 온 사람을 생각해보자.

모든 책을 다 읽고 오는 게 아니라, 사서에게 "금형 ECO 관련 자료 주세요"라고 물으면 사서가 서가에서 관련된 책만 몇 권 골라준다.

RAG는 AI판 도서관 사서다. 우리 문서 전체를 미리 "의미" 단위로 색인해두고, 질문이 들어오면 의미가 비슷한 부분만 빠르게 찾아서 AI에게 건네준다.

RAG, 장단점까지 정리한다

RAG는 한 문서·한 구간의 근거를 빠르게 찾지만, 여러 문서의 관계를 따라가는 질문에는 약하다

장점

장점 — 구축이 빠르다.

사서를 새로 교육하듯 문서를 넣기만 하면 되고, 문서가 수백~수천 건이어도 잘 확장된다.

단점

단점 — "이 ECO가 영향을 주는 다른 부품은?" 같은 문서 간 관계를 묻는 질문에는 약하다.

사서가 책 한 권씩은 잘 찾아줘도, 책들 사이의 연결고리까지 그려주지는 못하는 것과 같다.

지식그래프(GraphRAG)란 무엇인가

문서 속 "누가 누구와 연결되는지"를 그림으로 만들어두는 방식

비유로 설명하면

형사 드라마에서 수사관이 화이트보드에 용의자·증거·장소를 실로 연결해놓은 "인물관계도"를 떠올려보자.

지식그래프는 문서에서 부품·ECO·담당자·도면 같은 개체를 뽑아내고, 이들 사이의 관계를 선으로 이어둔 인물관계도다.
그래서 "이 부품과 연결된 다른 ECO는?"이라는 질문에 선을 따라가듯 바로 답할 수 있다.

지식그래프, 장단점까지 정리한다

GraphRAG는 관계 설명에 강하지만 구축·검증 비용이 들며, 환각을 없애는 장치는 아니다

장점

장점 — 관계형 질문에 강하다. "이 부품과 연결된 다른 ECO는?"
같은 질문에 선을 따라가듯 바로 답한다.

단점

단점 — 관계도를 그리는 작업(스키마 설계·추출) 자체에 시간과 비용이 든다.
형사가 화이트보드에 실을 처음부터 다 걸어야 하는 것처럼, 관계도를 처음 그리는 수고가 RAG보다 훨씬 크다.

RAG vs 지식그래프, 나란히 놓고 본다

둘 다 근거를 강화하지만 환각을 없애지는 않는다 — 질문 구조와 운영 비용으로 선택한다

RAG · 벡터 검색

문서를 의미로 찾는다

찾는 방식: 의미가 비슷한 문서 조각을 검색

잘 맞는 질문: 한 문서·한 구간에서
답을 찾는 질문

강점: 빠른 구축·쉬운 확장
약점: 문서 사이 관계 추적

지식그래프 · GraphRAG

문서를 관계로 연결한다

찾는 방식: 개체와 관계를 따라 근거 탐색

잘 맞는 질문: 여러 문서·부품·변경의
연결을 묻는 질문

강점: 관계 설명·영향 분석
약점: 스키마·추출·검증 비용

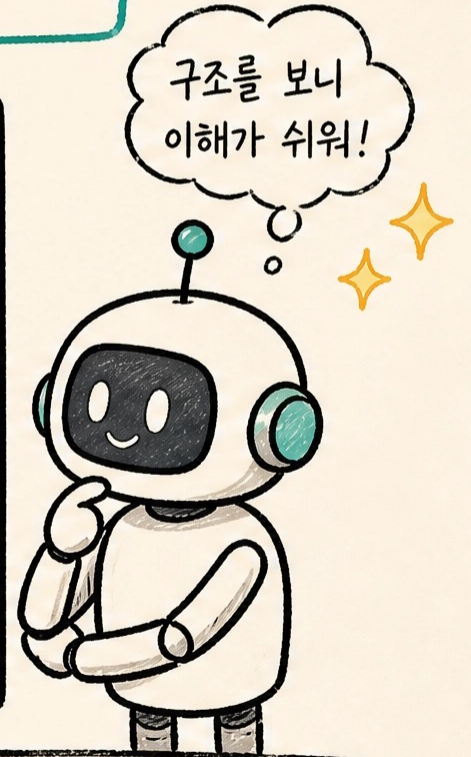
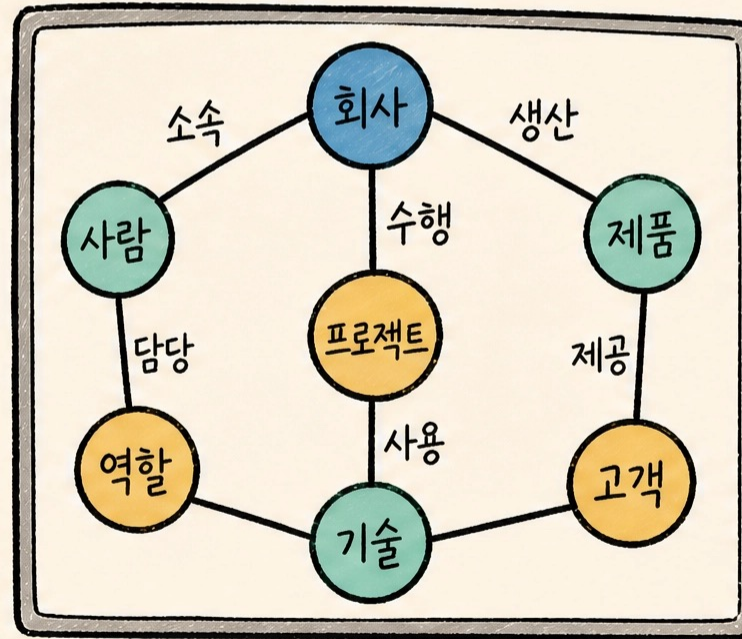
RAG vs 지식그래프

≡ 검색 vs 구조화 ≡

RAG
(Search-based)



Knowledge Graph
(Structured)



지식자산화는 6단계로 시작하고, 그래프는 필요할 때 확장한다

원본 보존 → 본문+메타데이터 → 표준 구조 → 관계 연결 → 재사용 목적 → 낡음·오류 관리

원본을 지키고 맥락과 관계를 붙여 다시 쓸 수 있는 지식으로 바꾼다



우리 팀은 RAG로 시작할까, 지식그래프로 시작할까

기본은 RAG로 시작하고, 관계 질문과 운영 여력이 모두 확인될 때 GraphRAG를 검토한다

Q1

답이 한 문서·한 구간 안에 있는가?



Q2

여러 문서의 부품·ECO·도면 관계를 따라가야 하는가?



Q3

스키마·추출·검증·유지 비용을 감당할 수 있는가?

결론

Q2와 Q3가 모두 ‘그렇다’면 GraphRAG 확장을 검토한다.

그 외에는 Markdown 표준화와 RAG부터 시작한다.

관계 질문이 실제로 반복될 때 그래프를 추가해도 늦지 않다.

[-] 2026+Alnative+AX	83
[-] _Templates	8
[-] 00. Inbox	1
[-] 01. Articles	27
[-] 02. Papers	0
[-] 03. Transcripts	6
[-] 04. Clippings	28
[-] Web Clipper	1
[-] 01. Projects	61
[-] ax.allrounder.im	3
[-] Commander Marketing AI Agent 활용서 2026	3
[-] _scripts	0
[-] __pycache__	0
[-] 01_목차별_MD	52
[-] 02_시나리오별_플레이북	17
[-] 03_템플릿	6
[-] 04_HTML_슬라이드_반영	6
[-] 06_완성_책_원고	1
[-] 08_이미지_원고삽입용	0
[-] 02. Areas	0
[-] 01. AI 자동화 & AX 전환	3
[-] 01. 전환 전략 & 로드맵	0
[-] 02. 조직별 구현 사례	0
[-] 03. 도구 & 플랫폼 평가	0



STEP 1

오늘 부터 git repo를 기반으로 실제 작업을 진행합니다

https://github.com/saewookkangboy/202608_sec_gumi/tree/main/mx-agentic-ai-day2-knowledge-harness

그림과 같이 폴더 구조를 가져갈 겁니다

/ mx-agentic-ai-day2-knowledge-harness /

사용법

1. 레포 루트에서 `cd mx-agentic-ai-day2-knowledge-harness` 후 Claude Code를 열어요.
2. Day 1 산출물인 `../mx-agentic-ai-day1-prd/docs/prd.md` 를 그대로 참고해요. 이 파일은 읽기만 하고 고치지 않아요.
3. 아래 [예제] 프롬프트를 순서대로 하나씩 붙여넣어요. 대괄호 [] 안만 본인 상황에 맞게 바꾸면 돼요.
4. 단계를 건너뛰거나 한꺼번에 시키지 마세요. 뒤 단계가 앞 단계의 결과를 그대로 쓰거든요.

저장소 구조

```

202608_sec_gumi/
├── CLAUDE.md
├── mx-agentic-ai-day1-prd/
│   └── docs/prd.md
└── mx-agentic-ai-day2-knowledge-harness/
    ├── README.md
    ├── docs/socratic-md-reading-prompt.md
    ├── knowledge/
    │   ├── eco/ECO-*.md
    │   ├── WIKI.md · catalog.json
    │   └── 001-[제목].md ...
    ├── relations.json
    ├── eval-top3.md
    ├── eval/
    │   ├── data/raw/
    │   └── _sandbox/
    └── scripts/ · tests/
  
```

- # 레포 공통 하네스 (오늘 갱신)
- # Day 1 산출물 (읽기 전용 참조)
- # 이 문서
- # 폴더 전체 .md 리딩 프롬프트 (복사용)
- # 참조 시나리오 지식 (이미 포함, 원본 잠금)
- # 인덱스
- # (선택) 팀 PRD용 추가 지식
- # GraphRAG 관계 (2-hop, 오늘 작성)
- # Top-3 검색 평가 (오늘 작성)
- # 참조 평가 세트
- # 원본 입력 (수정 금지)
- # 가상 테스트 전용
- # 정규화·검색·검증 ([선택] 참조 트랙)

예제 (오늘 쓸 프롬프트 — 순서대로)

Tip — 프롬프트에서 '(해당 git repo를 붙여넣고) 폴더 구조를 동일하게 생성해주세요' 라고 입력해보셔도 됩니다

STEP 2

하네스 엔지니어링을 위한 주요 .md 생성을 진행합니다

DAY1 에서 작성된 AI PRD 최종 문서를 기반으로

CLAUDE.md,

AGENTS.md,

plan.md,

progress.md,

decisions.md,

eval-top3.md

를 생성합니다.

ni / mx-agentic-ai-day2-knowledge-harness / docs / socratic-md-reading-prompt.md

rding in socratic-md-reading-prompt

(97 loc) · 7.42 KB

Day 2 · 전체 .md 생성 기반 리딩 프롬프트

레포 경로: mx-agentic-ai-day2-knowledge-harness/docs/socratic-md-reading-prompt.md

Day 2 폴더의 .md 25개를 한 번에 한 질문 으로 출으면서, 각 문서가 AI PRD Canvas 몇 번 칸을 지탱하는지 스스로 답해 보는 프롬프트입니다.

항목	내용
실행 위치	cd mx-agentic-ai-day2-knowledge-harness 후 Claude Code
선행 조건	Day 1 AI PRD 첨부 필수. 없으면 프롬프트가 게이트(G1~G5)에서 멈춰요
대상	.md 25개 (README.md 와 이 파일은 제외)
부작용	없음 — 읽기 전용입니다. 파일을 만들거나 고치지 않아요
산출물	채팅 답변(파일 지도 + Canvas 1~10 커버리지 표)

PRD를 첨부하지 않으면 동작하지 않습니다. 아래 중 하나로 준비해 주세요.

- Claude Code에 docs/prd.md (우리 팀 PRD)를 첨부하거나
- ../mx-agentic-ai-day1-prd/docs/prd.md 에 우리 팀 PRD가 저장되어 있어야 해요 (레포 기본 예시인 권적요청 초안봇 이면 프롬프트가 "이게 우리 팀 PRD가 맞나요?"를 먼저 되물어요)

복사용 프롬프트

Tip — 해당 작업을 하는 것으로 하네스의 기본적인 Rule 은 마련됩니다



좋은 지식 자산인지 확인하는 체크리스트

- ☐ 요약 결과가 원본 문서의 어느 부분에서 나왔는지 알 수 있는가(출처 추적 가능)
- ☐ 문서에 없는 정보를 에이전트가 지어내지 않았는가
- ☐ 결과를 팀 저장소의 정해진 위치(예: knowledge/)에 저장했는가
- ☐ 다른 팀원이 같은 파일을 열어봐도 이해할 수 있는 형식인가

코드도 머리도 잠시 멈춤
점심시간 · 휴식



60분 뒤에 다시 만나요

3

Day 2 · Module 3 ·

13:00-16:00

에이전트는 어떤 환경 안에서 일하는가

하네스 엔지니어링 — AGENTS.md로 행동 지침 통제

매번 같은 말을 반복하고 있지 않은가

"이 폴더는 건드리지 마", "이 형식으로 저장해" — 매 대화마다 이 말을 새로 하고 있다면 하네스가 없는 것이다

비유로 설명하면

신입사원이 매일 출근할 때마다 사수가 "이 서버는 건드리지 마세요",
"보고서는 이 양식으로 쓰세요"를 처음부터 다시 설명한다면 얼마나 비효율적일까?

그래서 회사는 "신입사원 온보딩 매뉴얼"을 만든다.
한 번 문서로 정리해두면, 누가 와도 그 문서만 보고 일할 수 있다.
AGENTS.md가 바로 에이전트를 위한 온보딩 매뉴얼이다.

하네스 엔지니어링(Harness Engineering)이란 무엇인가

하네스는 AI를 더 똑똑하게 만드는 장치가 아니라, 해야 할 일과 하지 말아야 할 일을 환경으로 고정하는 장치다

"지시서·구조적 제약·검수·기억·권한·복구·관측을 묶어, 에이전트의 행동을 예측 가능하게 만드는 일."

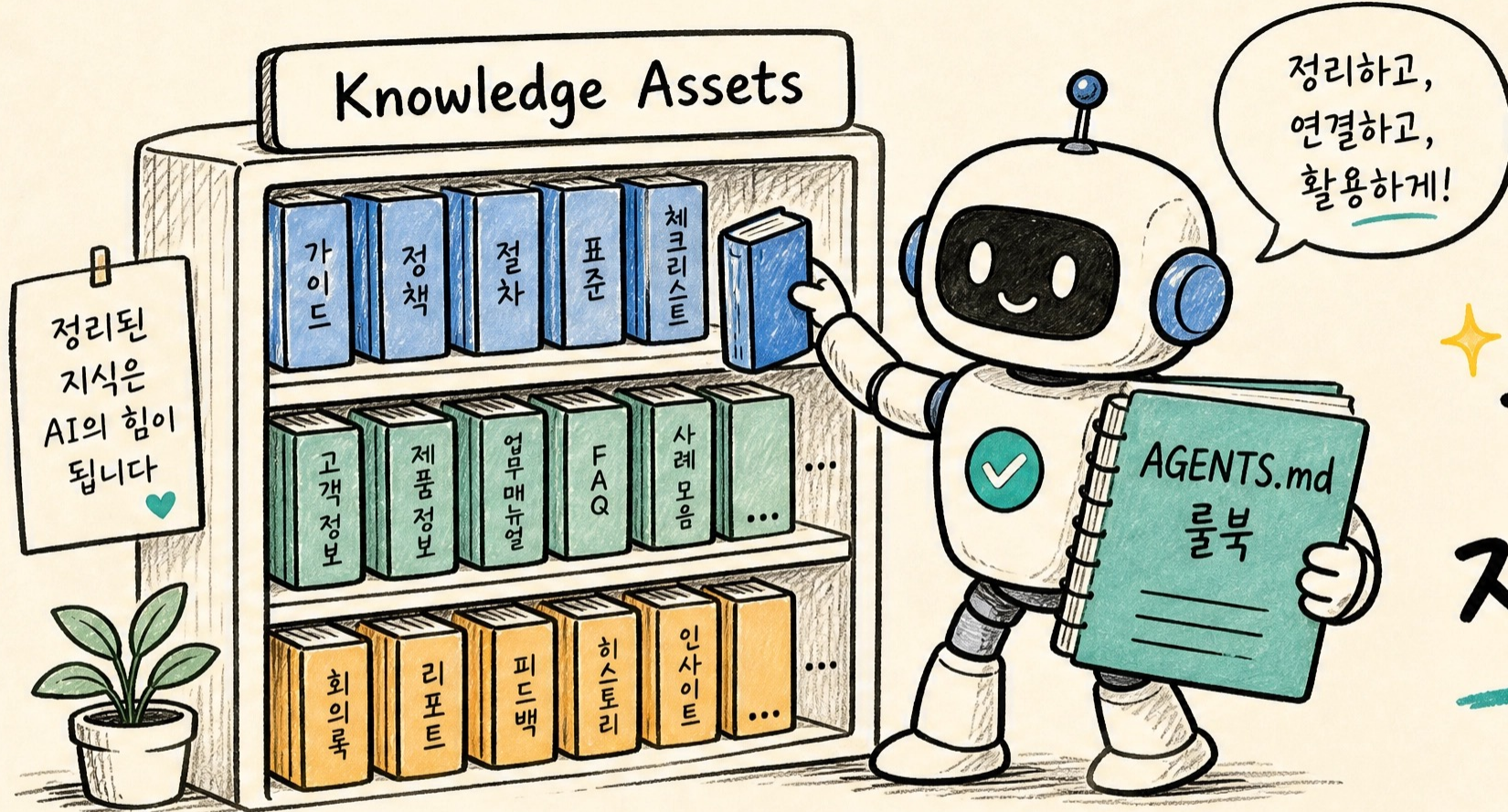
Harness를 Loop로 착각하면

권한이 잘못된 채로 반복 횟수만 줄이려다
근본 원인(과도한 자동 승인)을 놓친다

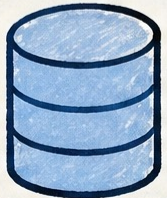
Loop를 Harness로 착각하면

종료 조건이 없는데 도구 권한만 좁히면
무한 반복 비용은 그대로 남는다

AI가 일을 더 잘하게 하려면

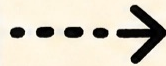


하네스와 지식 자산화 설계



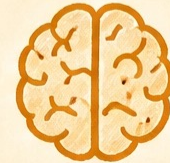
데이터

다양한 원천의 정보를
수집하고 구조화합니다



규칙

명확한 규칙과 가이드를
정의하고 일관되게 적용합니다



기억

경험과 지식을 축적하고
필요할 때 똑똑하게 꺼내 씁니다

하네스를 이루는 7가지 요소

지시 문서만으로는 부족하다 — 제약·검수·기억·권한·복구·관측이 함께 작동해야 한다

하네스는 지시를 넘어 제약·검수·기억·복구까지 묶은 실행 환경이다



지시 문서

목표·출력·금지·완료 기준



구조적 제약

원본 보호·승인·자동 검사



피드백 루프

테스트·체크리스트·검토



지식 저장소

결정·실패·교훈을 기억



도구와 권한

할 수 있음과 해도 됨 구분



상태와 복구

중단 후 이어서 작업



관측성과 정리

로그·남은 규칙·임시물 관리

7가지 요소, 회사 생활에 빛대면

낯선 용어가 많다면 이 표로 감을 잡는다

요소	회사 생활 비유
지시 문서	업무지시서 — 목표·출력·금지·완료 기준
구조적 제약	출입문·결재선 — 규칙 위반을 시스템으로 차단
피드백 루프	중간 검토 — 테스트·체크리스트·리뷰로 즉시 교정
지식 저장소	업무 위키 — 결정·실패·교훈을 조직이 기억
도구와 권한	사내 시스템 권한 — 할 수 있음과 해도 됨을 구분
상태와 복구	인수인계표 — 중단된 지점부터 다시 시작
관측성과 정리	작업 일지·정리일 — 로그를 남기고 낡은 규칙을 제거

Harness Engineering: 안전한 작업 환경 설계 ≡

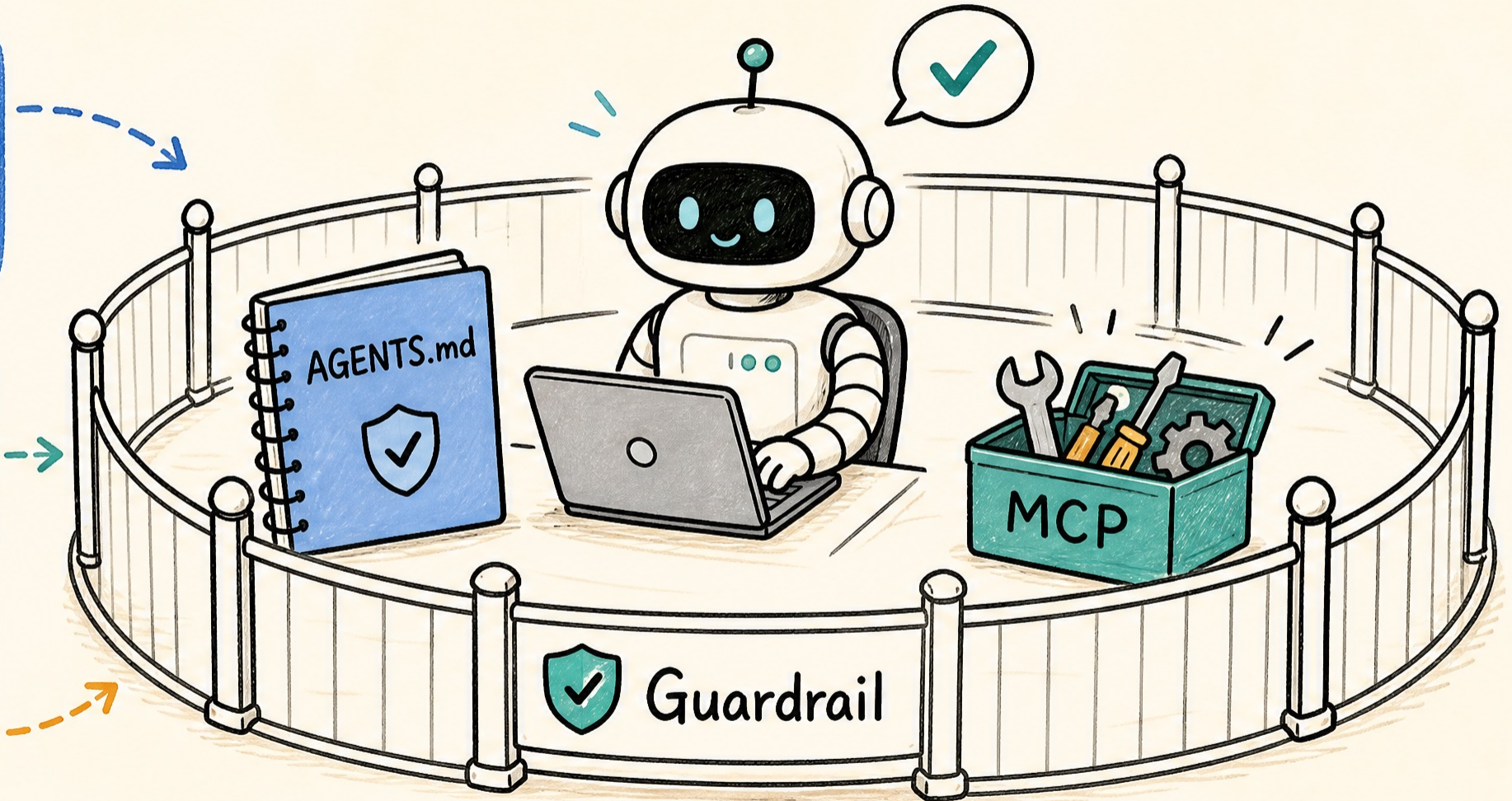
1 맥락 정의
(Context)



2 도구 제공
(Tools)

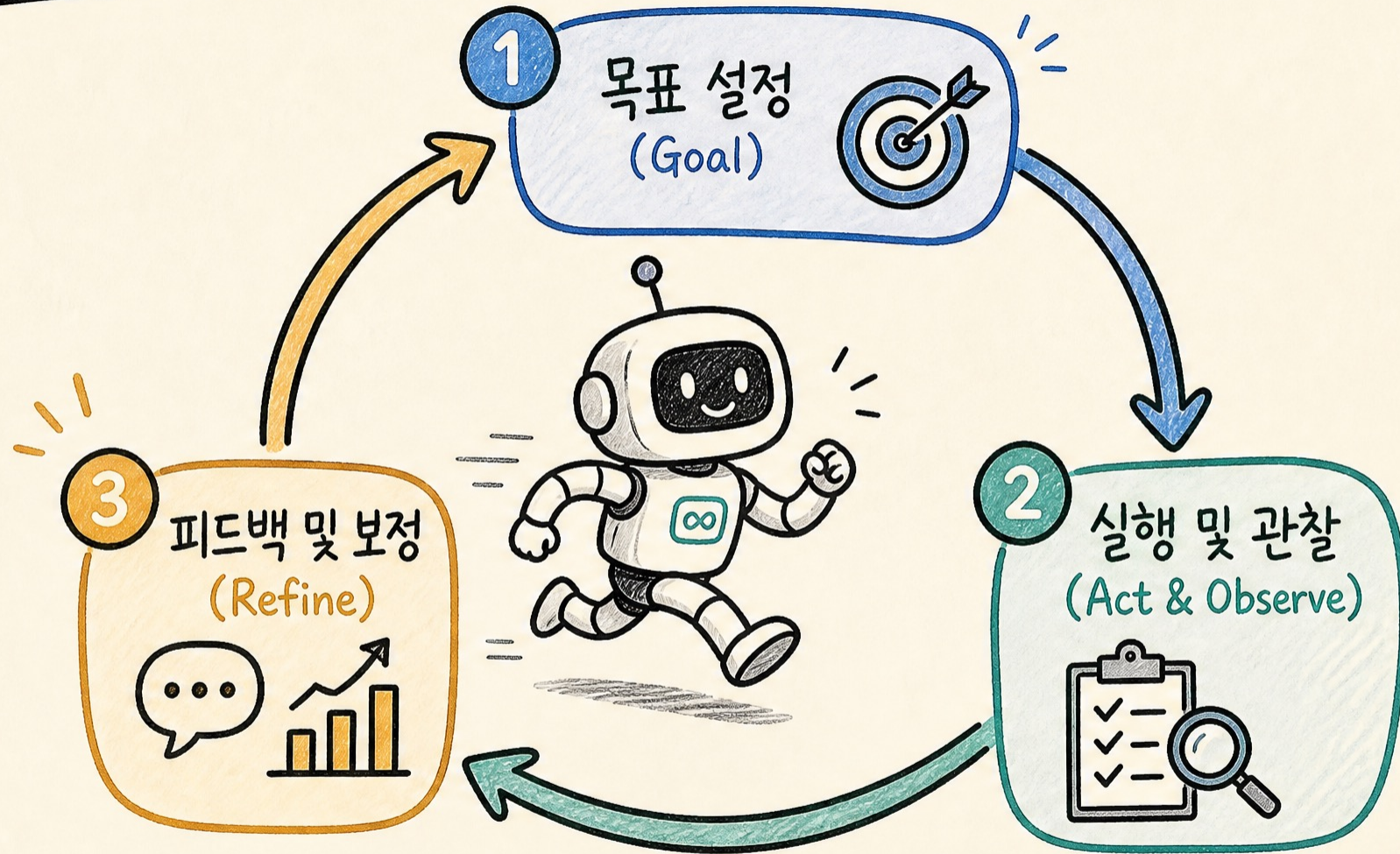


3 안전 장치
(Safety)



≡ 통제 가능한 에이전틱 워크스페이스 ≡

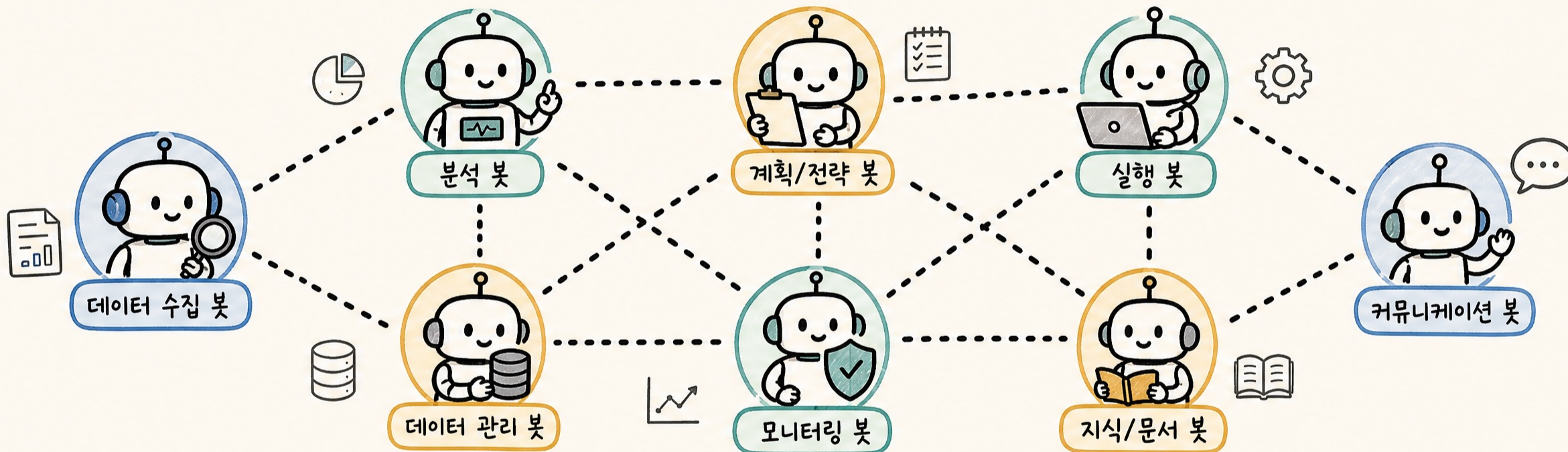
Loop Engineering: 실행과 개선의 반복



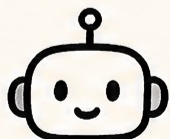
☆ 단일 에이전트의 완결성 확보 ☆

Graph Engineering: 복잡한 연결의 구조화

☆ ☆ 멀티 에이전트 협업 체계 구축 ☆ ☆



1 역할 정의 (Node)



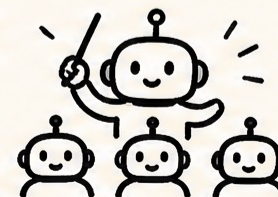
각 에이전트의 역할과 전문성을 정의하여 노드로 구성합니다.

2 흐름 설계 (Edge)



에이전트 간의 연결과 정보 흐름을 설계하여 엣지로 구성합니다.

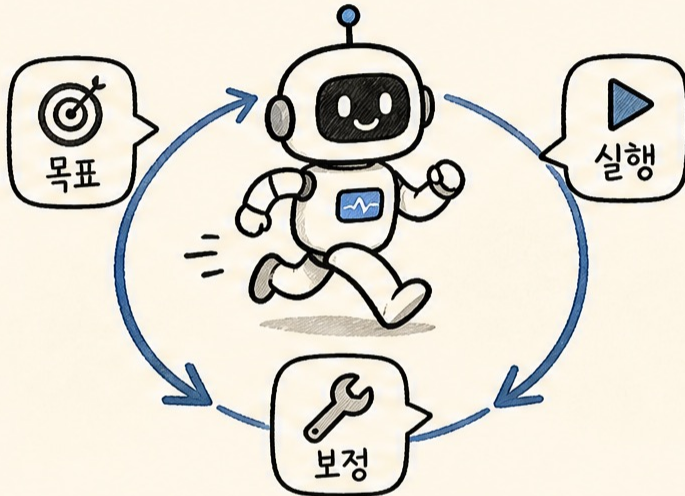
3 전체 오케스트레이션 (Orchestrate)



설계된 그래프를 기반으로 에이전트들이 유기적으로 협업하여 목표를 달성합니다.

3대 에이전틱 엔지니어링 기법 비교 가이드

1 Loop Engineering



Key

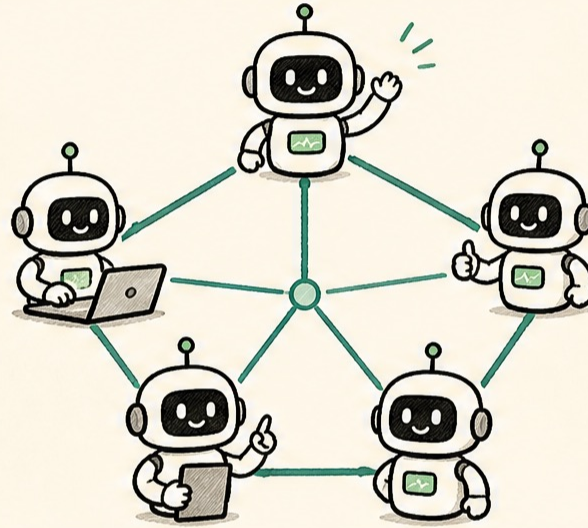
반복과 개선



Guide

목표 → 실행 → 보정

2 Graph Engineering



Key

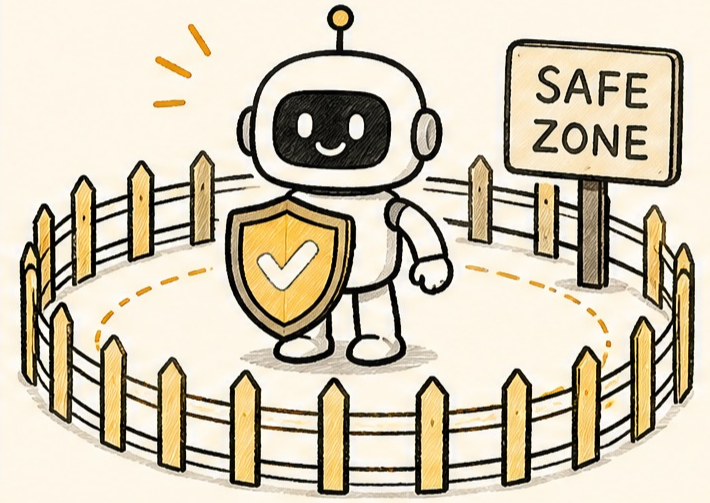
연결과 협업



Guide

역할 → 흐름 → 조율

3 Harness Engineering



Key

통제와 안전



Guide

맥락 → 도구 → 보안



상황에 맞는 최적의 기법 선택



AGENTS.md — 에이전트의 행동 지침서

프로젝트 저장소에 두는 규칙 파일. 에이전트가 작업을 시작할 때마다 자동으로 읽는다

정의

무엇을 담는가

프로젝트 배경, 폴더 구조 규칙
코드/문서 작성 컨벤션
하지 말아야 할 행동(금지 목록)
커밋·검증 절차

한 파일 안에 "이 프로젝트에서
일하는 규칙"을 전부 모아둔다

효과

왜 필요한가

반복 지시를 줄이고 팀 규칙을 유지한다
목표·출력·금지·완료 기준을 명시한다

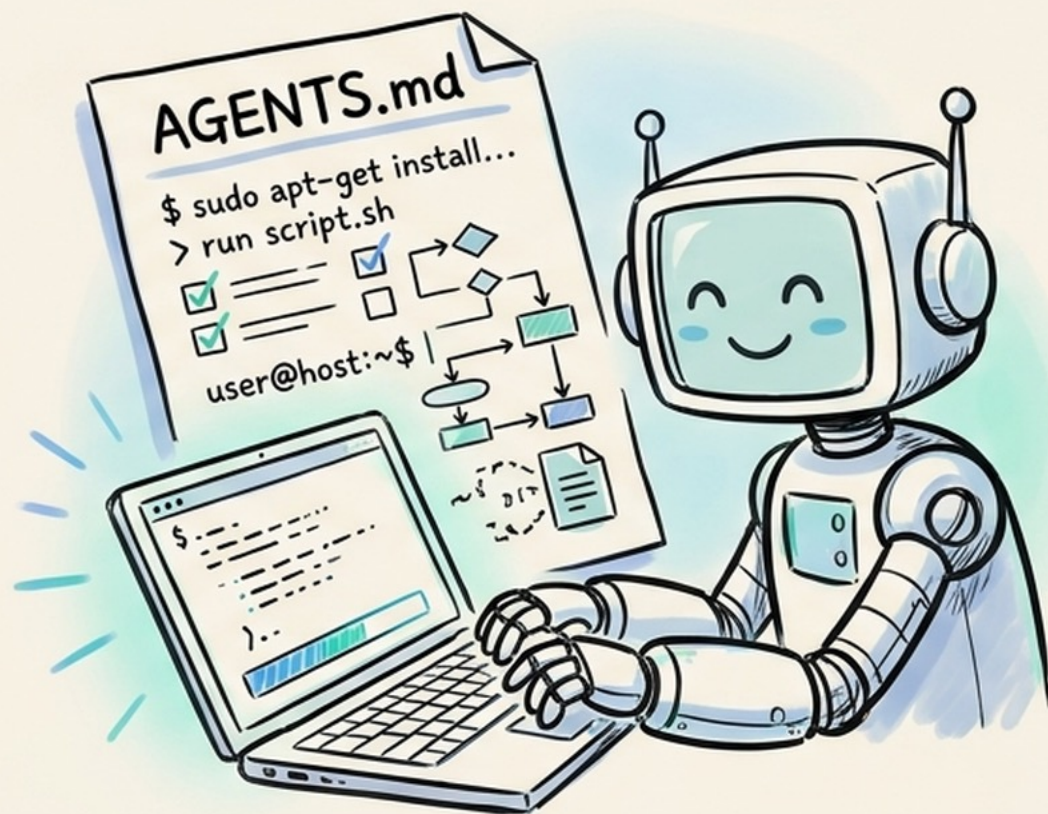
단, 문서만으로는 충분하지 않다
권한·자동 검사·승인·로그 같은
구조적 제약과 함께 작동해야 한다

README는 사람을 위한 문서

AGENTS.md는 에이전트 설명서



사람용 소개 · 배경과 맥락



에이전트용 지침 · 명령과 규칙

무엇을 적어야 하나

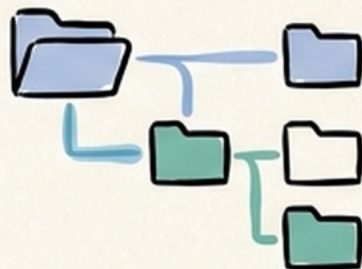
AGENTS.md 핵심 구성 6가지



실행 명령어



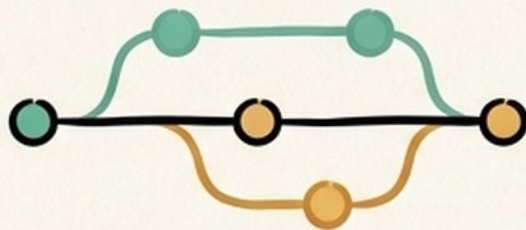
테스트 방법



프로젝트 구조



코드 스타일 예시



Git 워크플로우



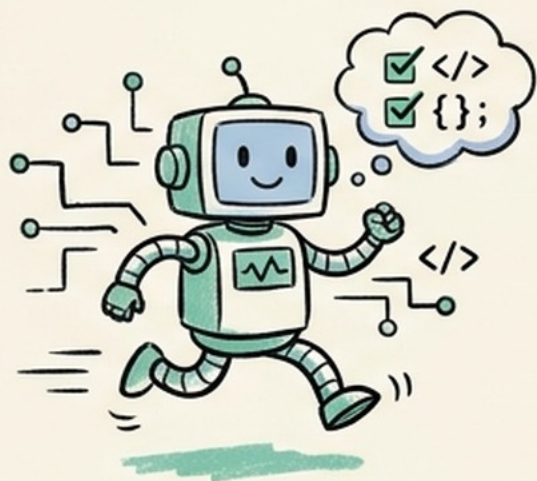
금지 경계

에이전트에게 선을 그어라

3단계 경계 설정



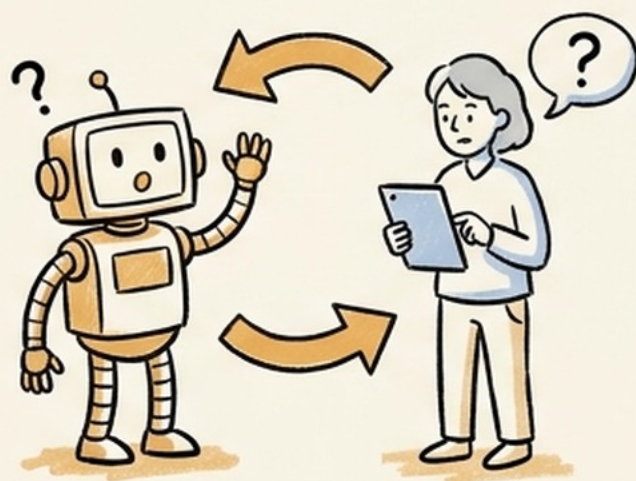
항상 실행



테스트 실행 · 포맷 적용



먼저 확인



스키마 변경 · 배포



절대 금지



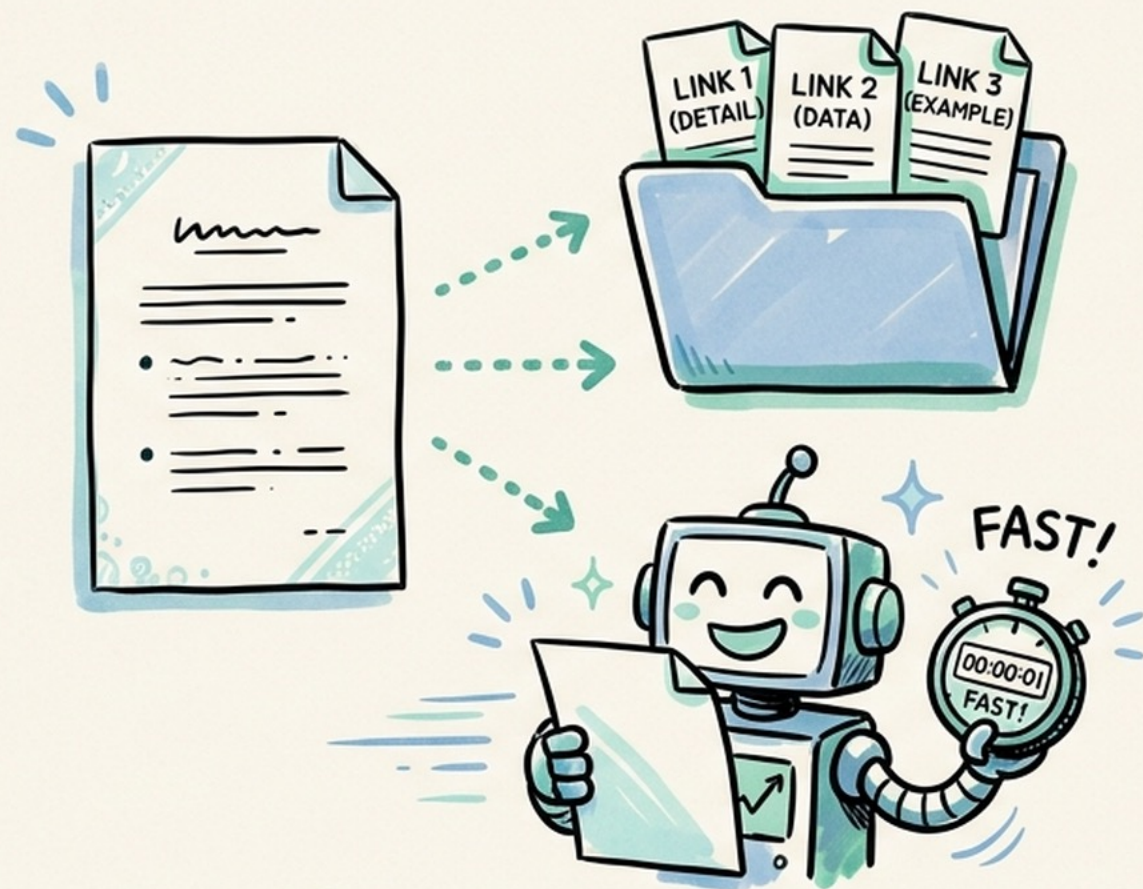
비밀키 커밋 · 데이터 삭제

길수록 좋은 게 아니다

짧고 정확하게 60~300줄



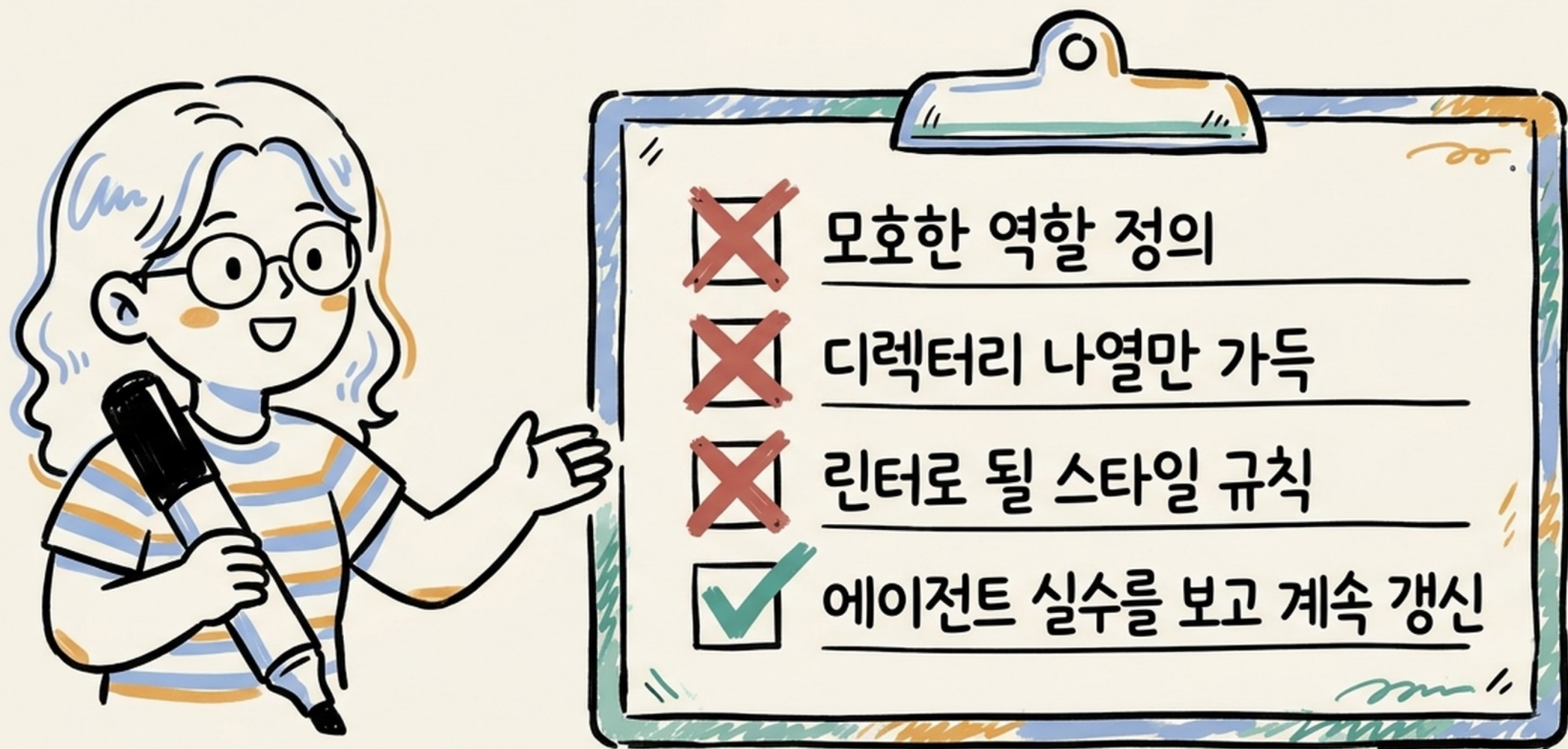
자동 생성 문서 · 토큰만 늘어난다



핵심만 본문 · 상세는 링크로

이렇게 쓰면 실패한다

AGENTS.md 체크리스트



STEP 1

AGENTS.md가 정상적으로 생성되었는지 확인한다

https://github.com/saewookkangboy/202608_sec_gumi/blob/main/mx-agentic-ai-day2-knowledge-harness/AGENTS.md

기본적인 repo skill 로 되었지만
각자 만든 AGENTS.md 생성된 것이
안전규칙, 작업 상태를 모두 반영되었는지
확인합니다

Repository Instructions

목적

합성 금형 ECO 문서를 추적 가능한 지식 자산으로 변환하고 검색 품질을 검증한다. 팀 PRD 연속형 실습은 `relations.json` · `eval-top3.md` · (선택) `knowledge/001-*.md` 를 사용한다.

안전 규칙

- `data/raw/` 의 원본 파일을 수정·삭제·이동하지 않는다.
- 생성 결과는 `knowledge/`, `relations.json`, `eval-top3.md`, `outputs/`, `reports/`, `_sandbox/` 에만 기록한다.
- 문서에 없는 값은 추정하지 말고 `UNKNOWN` 으로 기록한다.
- 가상 테스트는 `_sandbox/` 에서만 한다.
- 변경 전 대상 파일 목록을 먼저 제시한다.
- 완료 전 `eval-top3.md` 확인과 (선택) `python3 scripts/validate_repo.py` 를 실행한다.

작업 상태

- 계획은 `plan.md`, 진행 상태는 `progress.md`, 선택 근거는 `decisions.md` 에 기록한다.
- 이미 완료된 단계는 재실행하지 말고 다음 미완료 단계부터 이어간다.

Claude Code는 같은 폴더의 `CLAUDE.md` 가 이 파일을 가져온다.

Tip — 스킬은 `.agents/skills/<name>/SKILL.md`에 저장되어 저장소를 복제한 팀원이 함께 사용한다.

자동 테스트 — 하네스와 검색이 실제로 작동하는가

- 1 STEP 3-1** 새 대화창을 연다 (지금까지의 대화 기록이 안 보이는 상태)
- 2 STEP 3-2** "우리 AGENTS.md를 읽고 자기소개해주세요"라고만 요청한다
- 3 STEP 3-3** 에이전트가 프로젝트 배경·금지 행동을 정확히 요약하면 통과
- 4 STEP 3-4** 빠진 내용이 있으면 AGENTS.md에 그 부분을 더 명확하게 다시 적는다

Tip — 새 대화창에서도 규칙이 살아있는지 확인하는 것이 하네스 검증의 핵심이다



좋은 하네스인지 확인하는 체크리스트

- ☐ 목표·출력·금지·완료 기준이 명확한가?
- ☐ 원본 보호와 승인 규칙이 시스템으로 강제되는가?
- ☐ 테스트·체크리스트·리뷰가 즉시 피드백을 주는가?
- ☐ 중단 후 상태·근거·승인 지점에서 복구할 수 있는가?
- ☐ 로그를 남기고 낡은 규칙·임시 파일을 정리하는 주기가 있는가?

4

Day 2 · Module 4 ·

16:00–18:00

긴 작업은 어떻게 기억을 유지하는가

컨텍스트 실무 — 외부 메모리와 크로싱 메모리 관리

대화창을 닫으면 에이전트는 기억을 잃는다

어제 나눈 대화, 오늘은 기억하지 못한다 — 긴 자동화 작업일수록 이 문제가 치명적이다

비유로 설명하면

매일 아침 기억을 잃는 주인공이 나오는 영화를 떠올려 볼까요?

그 주인공은 몸에 문신을 새기고 사진에 메모를 적어 "어제까지 무엇을 했는지"를 스스로에게 알려준다.

에이전트도 대화창이 바뀌면 기억을 잃는다.

그래서 우리는 plan.md·progress.md·decisions.md라는 "메모"를 남겨, 에이전트가 그것만 읽어도
이어서 작업할 수 있게 만든다.

메모리 레이어 3분리

대화·조직 기억·검색 지식을 분리하면 중단 후에도 맥락과 책임 근거를 복구할 수 있다

세 가지 기억을 분리하면 중단 후에도 맥락을 복구할 수 있다



컨텍스트와 외부 메모리

★ 잊지 않는 에이전트 ☆

컨텍스트 윈도우
(현재 집중 영역)

- 현재 단계: 4. 교각 건설
- 사용 자재: 철골, 콘크리트
- 길이: 120m
- 다음 단계: 보 구조 설치
- ...

기억해야 할 것들
(금방 잊을 수 있어요!)



설계도



자재 사양



주의사항



결정 사항

...

긴 프로젝트 (긴 작업)

- 1 설계 및 계획
- 2 자재 준비
- 3 기초 공사
- 4 교각 건설
- 5 보 구조 설치
- 6 다리 상판 설치
- 7 마무리 및 점검
- ...

안전 제일!

외부 메모리 ☆
(External Memory)

기록하기!

- ☒ 프로젝트 개요 및 목표
- ☐ 설계 세부 정보
- ☐ 자재 정보 및 공급처
- ☐ 리스크 및 주의사항
- ☒ 중요한 결정 사항
- ☐ 작업 기록 및 로그
- ...

중요한 정보를
기록해 두고,
필요할 때
꺼내 확인!

마크다운 파일로 장기 작업 상태를 기록한다

대화가 끊기거나 다음 날 이어서 작업해도, 파일을 읽으면 에이전트가 맥락을 복원한다

파일	역할	기록 내용
plan.md	작업 계획	이번 자동화 파이프라인이 무엇을 하는지, 단계별 순서
progress.md	진행 상태	어디까지 완료됐는지, 다음에 무엇을 할지
decisions.md	결정 사항	왜 이 방식을 선택했는지 — 나중에 되돌아볼 근거

핵심 정리

이 세 파일은 하네스의 ‘지식 저장소’와 ‘상태·복구’를 마크다운으로 구현한 최소 예시다.

☆ 지식 자산화와 하네스 이야기 ☆



☁ 엉망인 데이터를 ➡ 지식 자산으로 만들고, ➡ 하네스에서 가치 있게 활용하자! ☆

세션 끊김을 일부러 시뮬레이션해본다

이 테스트를 통과해야 "진짜로 기억이 유지되는지" 확인할 수 있다

비유로 설명하면

영화 속 기억상실 주인공이 정말 메모만 보고 다음 행동을 이어갈 수 있는지 시험해보는 장면과 같다.
우리도 지금 대화를 "일부러 끊고" 다시 시작해본다.

1

지금 대화창을 닫거나 새 대화창을 연다



2

"progress.md를 읽고
이어서 진행해주세요"라고만 말한다



3

에이전트가 스스로 맥락을 복원해 이어가면
성공



완주 체크 — 장기 자동화 시나리오

- | | | |
|---|---------|--|
| 1 | CHECK 1 | plan.md에 이번 시나리오의 목표와 단계가 적혀 있는가 |
| 2 | CHECK 2 | progress.md가 각 단계마다 갱신됐는가 |
| 3 | CHECK 3 | 새 대화를 열어도 progress.md만으로 이어서 작업이 가능한가 |
| 4 | CHECK 4 | decisions.md에 왜 이 방식을 선택했는지 최소 1줄 이상 있는가 |

오늘 배운 3가지

1

지식 자산화

지식자산화는 원본·메타데이터·구조·관계·재사용 목적·최신성을 함께 관리한다

2

하네스 설계

하네스는 지시·제약·검수·기억·권한·복구·관측을 묶어 행동을 예측 가능하게 만든다

3

컨텍스트 실무

plan/progress/decisions 파일은 결정과 상태를 남겨 중단 후에도 작업을 복구한다

Day 2 인계 — `request_handoff.py --day 2` → 사람 승인 → `check_day_gate.py --enter-day 3`

지식 자산화 · 하네스 엔지니어링 실무

DAY 3 에서 뵙겠습니다

오늘도 애쓰셨습니다.